Several Path-Planning Algorithms of a Mobile Robot for an Uncertain Workspace and their Evaluation

Hiroshi Noborio
Department of Precision Engineering
Osaka Electro-Communication University
Hatsu-Cho 18-8, Neyagawa,
Osaka 572, JAPAN

ABSTRACT: Lumelsky's algorithms succeed in selection of a collision-free and deadlock-free path from a start point to a goal point in an uncertain workspace. In this paper, we study these mechanism and at last find a sufficient condition to design such good algorithms. Based on the sufficient condition, we can construct flexibly similar algorithms with several qualities to generate collision-free and deadlock-free paths in an unknown workspace. Some of them are better than Lumelsky's algorithms in an unknown workspace with simple shaped obstacles with respect of the path length measured by the Euclidean distance, and others of them are better than Lumelsky's algorithms in a dispersed workspace with complex shaped obstacles with respect of the Euclidean path length. Moreover, this paper shows that Lumelsky's algorithms are superior to my algorithms in a complicated shaped unknown workspace with respect of it. Finally, these characters are ascertained by geometrical properties and experimental results.

I. INTRODUCTION

Intelligent mobile robot requires a basic function such that it arrives at a goal point surely in an arbitrary shaped workspace while avoiding several obstacles. In robotics, the function is called and studied as the path-planning, in other words, the obstacle avoidance, and several path-planning algorithms have been already presented. However, almost all algorithms deal with only a fixed workspace whose obstacle shape and location are already known locally and globally [1]-[4]. Even if every mobile robot developed already uses some sensors skillfully, it is difficult for the robot to acquire complete information about obstacle shape and location in the workspace. Observation of this, the path-planning algorithm presented already cannot support any mobile robot intelligently. Fortunately, Lumelsky presents two path-planning algorithms for dealing with an uncertain workspace [5],[6]. By keeping one of the algorithms, the mobile robot is able to have an intelligence in every workspace with regard to the obstacle avoidance even if it does not know any information about obstacle shape and location.

In this paper, we research why Lumelsky's algorithms select a reasonable collision-free and deadlock-free path toward a goal point even in an uncertain workspace, and consequently we find a sufficient condition to design such good path-planning algorithms. Based on the sufficient condition, we can construct easily and flexibly several good algorithms with different characters for an unknown workspace, which are different from Lumelsky's algorithms. Some of the algorithms select a shorter deadlock-free and collision-free path in an uncertain workspace with simple shape, and others of them select such a more smooth path in a dispersed workspace with complex shape. Moreover, this article shows that Lumelsky's algorithms, which are satisfied with the sufficient condition, are superior to the other algorithms in a complicated workspace with complex shape.

Finally based on these characters, we try to fit one of my and Lumelsky's algorithms to some shaped workspace so as to cut the path short. These fitness is ascertained by geometrical properties and experimental results.

II. PATH-PLANNING ALGORITHM IN AN UNCERTAIN WORKSPACE

In this section, we explain three typical algorithms to generate intuitively reasonable deadlock-free and collision-free paths with different shape in an unknown workspace. First of all, we discuss a mobile robot and its workspace where the algorithm can play as follows:

<Workspace definition> In this research, we consider a two-dimensional workspace as follows: There are a lot of obstacles with arbitrary shape, which do not touch each other. The obstacle has a boundary whose length is of finite in the two-dimensional workspace. Needless to say, the number of obstacles is unrestricted in the workspace.

<Robot definition> In this research, we consider that a mobile robot is defined by a point, which can pass free between the obstacles in the above workspace. In general, the mobile robot do not have any information about the workspace. Nevertheless, the robot always knows its present position and also a given goal position in the workspace. Further, the robot can find a touch with some obstacle boundary by a touch sensor, vision and so on.

As a result, the robot can calculate its direction and distance toward the goal. Consequently, the robot makes one of the following three actions: move along a straight line toward the goal; move around an obstacle boundary; and stop. In a two-dimensional workspace, the robot selects either left or right direction to trace each obstacle boundary by means of the local sensor information.

If we select start and goal points which are not separated by an obstacle in the workspace, the proposed algorithms determine intuitively reasonable collision-free and deadlock-free paths surely. Otherwise, no algorithm selects such a path between start and goal points. In this case, my algorithms finish after they recognize automatically such a hopeless situation (Fig.1).

My algorithms control basically the mobile robot as follows: The robot always goes to the goal if it does not collide with any obstacle. If the robot hits an obstacle by a point H, it traces the obstacle boundary until a point L where:

(1) The robot can leave the obstacle to go to the goal without collision of the obstacle.

(2) The point L is closer than the point H against the goal.

Roughly speaking, since the robot leaves each obstacle from the point L which is nearer than the hit point H toward the goal, the robot approaches the goal even if it avoids an obstacle. Needless to say, the robot approaches the goal when it does not collide with any obstacle. As a result, the algorithms let the robot come near the goal in the

workspace and consequently the robot arrives at the goal surely (Fig.2).

Now, there are an infinite number of such points keeping the above two conditions around each obstacle. Therefore by flexible selection of the leave point L from the point set, we design easily several path-planning algorithms and can find some good algorithms from them. Lumelsky's algorithms belong to the algorithm set. In this section, we select three typical algorithms from the algorithm set and then compare them with Lumelsky's algorithms.

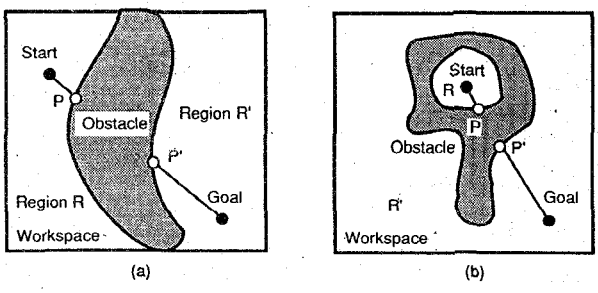

Fig.1 Start and goal points separated by an obstacle in a workspace

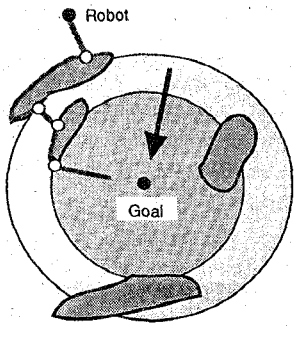

Fig.2 The decrease of the Euclidean distance d* (The decrease of obstacle boundaries where the robot can touch)

A. The Algorithm Class1

In this algorithm, the robot leaves each obstacle from the nearest point L against the goal on the path generated previously, where the robot can go to the goal without collision of the obstacle. As compared with other algorithms, the algorithm Class1 lets the mobile robot forbear to leave from each obstacle because the leave point L is defined by the closest point toward the goal where the robot stayed so far in the workspace. This character leads that the robot avoids each obstacle so as not to collide with the obstacle many times in almost all workspaces. Especially in a complicated shaped unknown workspace, the character is to be desirable and in result the algorithm Class1 can generate a shorter path.

The algorithm moves the robot, and then it hits several obstacles by the points H_i, i=1,2,..., and on the other hand, the robot leaves the obstacles by the points L_i, i=1,2,.... Initially, i=1; L_0=Start (See Fig.3).

1. From the point L_{i-1}, the mobile robot moves toward the goal along a straight line until one of the following occurs:

(a) The procedure stops with success if the robot reaches the goal.

(b) If an obstacle is encountered, then a hit point H_i is defined and go to Step 2.

2. Selecting one of right and left direction, the robot follows the obstacle boundary. If the robot reaches the goal, the procedure stops with success. Otherwise after having traversed the boundary until an arbitrary point Li, the robot leaves the obstacle to go toward the goal. The point Li is selected on the satisfaction with the following conditions:

(1) The robot can go to the goal without colliding with the obstacle.

(2) The Euclidean distance toward the goal is smaller than an Euclidean distance d*, which is the minimum of distances between one point along the previously generated path and the goal.

Then, return to Step 1 with the point Li.

3. If the robot cannot find the point Li and consequently returns to Hi, we recognize that the robot can not arrive at the goal geometrically in the workspace since start and goal points are separated by an obstacle in the workspace. Thus the algorithm stops with failure.

B. The Algorithm Class2

In this algorithm, the robot leaves each obstacle from the point L, which is the nearest point of all leave and hit points toward the goal on the path generated previously, and also where the robot can go to the goal without collision of the obstacle. Since the point L is located as a medium position between these points L selected by the algorithms Class1 and Class3 around the obstacle boundary, the robot leaves an obstacle earlier than the algorithm Class1. In result, the algorithm Class2 has a middle character between the algorithms Class1 and Class3.

In the algorithm Class2, the step 2(2) in the algorithm Class1 is replaced by the following step:

(2) The Euclidean distance toward the goal is smaller than an Euclidean distance d*, which is minimum of distances from hit and leave points on the previously generated path toward the goal.

C. The Algorithm Class 3

In this algorithm, the robot leaves each obstacle from the point L, which is the nearest point of all leave points toward the goal on the path generated previously, and also where the robot can go to the goal without collision of the obstacle. The robot finds the point L earliest around the obstacle in comparison with the other algorithms, and consequently the robot leaves from each obstacle earliest. This property is the most desirable in a cluttered workspace with simple shaped obstacles, but is not desirable in a complicated shaped workspace. Since the early remove smooths the path shape after the robot avoids the obstacle and the robot does not collide with each obstacle many times, the algorithm Class3 is good in the former case. On the other hand, the algorithm Class3 is not always good in the latter case because the smooth path is generated but the robot collides with the same obstacle a lot of times.

In the algorithm Class3, the step 2(2) is also replaced in the algorithm Class1 by the following step:

(2) The Euclidean distance toward the goal is smaller than an Euclidean distance d*, which is minimum of distances from leave points toward the goal along the generated path.

In these algorithms, we select the distance d* which decreases monotonously per avoidance of each obstacle, and in result the robot comes close to the goal and finally arrives at the goal. This proof of deadlock-free and collision-free is given in the next section. A decreasing pace of the distance d* becomes slow from the algorithm Class 1 to the algorithm Class 3. Therefore the robot leaves from each obstacle earlier, and consequently the path length and shape are flexibly changed.

D. Lumelsky's algorithms Bug1 and Bug2

Comparing my algorithm with Lumelsky's algorithms Bug1 and Bug2, we address them as follows:

Bug1: The robot always goes toward the goal. If it collides with an obstacle, the robot goes round the obstacle to determine the closest point for the goal, and leaves from the obstacle from the closest point to go toward the goal.

Bug2: The robot moves along the straight line between start and goal points. If the line intersects an obstacle, the robot traces the obstacle boundary from a point H where the line enters into the obstacle according to a given clockwise or counter-clockwise direction, and then the robot leaves the obstacle from a point L, where the line goes out the obstacle and simultaneously which is closer than the point H toward the goal.

These algorithms belong to my algorithm Class1 and Class2 respectively. That is, the leave point is selected as the nearest point along the generated path toward the goal in the algorithm Bug1. Thus the algorithm Bug1 is considered to be the special one in my algorithm Class 1. The algorithm Bug1 has an unique character that the robot never returns to an obstacle twice because the robot leaves the obstacle from only the closest point toward the goal, and this character is good for a complicated shaped workspace. In the

algorithm Bug2, the points H and L are monotonously closer toward the goal, and therefore the algorithm Bug2 is also considered to be the special one in my algorithm with the Class 2. The algorithm Bug2 also controls only leave points on the straight line between start and goal points, and the number of point is of finite. This finiteness is also good for a complicated shaped workspace since the algorithm Bug2 does not occur meaningless leave from an obstacle.

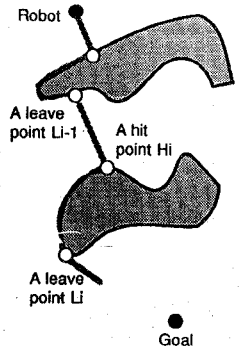

Fig.3 Hit and leave points on several obstacles.

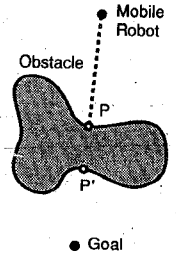

Fig.4 A closer point than a hit one H around the hitting obstacle

III. PROOF OF PATH GENERATION WITH DEADLOCK-FREE AND COLLISION-FREE

On the basis of the proposed algorithms, the mobile robot surely gets the goal while avoiding the obstacles. In this section, we first show that if start and goal are selected which are not divided by any obstacle in the workspace, the robot can find a point L and consequently leave each obstacle, otherwise, the robot can not find any point L and in result can not leave from the obstacle. Under these facts, we afford proof of the robot arrival to the goal.

[Theorem 1] If the robot going toward the goal touches an obstacle O at a point P, there is a closer point P' around the obstacle O, where the robot can go toward the goal without the collision.

(Proof) The segment PG intersects boundary of the obstacle O many times, and then the robot can leave behind it from the closest intersection point toward the goal illustrated in Fig.4. The proof is finished since the closest point is regarded as the point P'.

[Theorem 2] If the robot hitting an obstacle O at a point P can not leave behind the obstacle O, it divides start and goal points in the workspace.

(Proof) As shown in Theorem 1, there is a point P' around the obstacle O. However in this case, the robot can not reach the point P' even although it gets round the obstacle O. This occurrence indicates that the obstacle divides the points P and P' in the workspace. It also shows that the obstacle divides the workspace into the region R including the start and the region R' including the goal (See Fig.1). It completes the proof.

There is not any collision-free path between start and goal points if they are separated each other by some obstacle in the workspace. In the Step 3 of my algorithm, such a hopeless case is automatically judged in an uncertain workspace by checking whether the robot leaves behind each obstacle or not.

[Theorem 3] The robot comes close to the goal while touching several obstacles time after time. Then the robot arrives at the goal surely if the distance d* between the leave point and the goal is successively decreased.

(Proof) On the basis of Theorem 1, the robot can leave behind each obstacle so as to decrease the Euclidean distance d*. If it is so, regions around the obstacles are decreased where the robot can leave (See Fig.2). Furthermore based on the contraposition of Theorem 1, the robot can not collide with the obstacle where it can not leave. As a result, regions leaving behind the obstacles (those touching the obstacles) are monotonously decreased. Consequently, the robot leaves at worst from the closest point around the obstacles toward the goal and then arrives at the goal.

IV. ESTIMATION OF PATH LENGTH BY THE EUCLIDEAN DISTANCE AND ITS DEMONSTRATION BY AN EXPERIMENTAL RESULT

In this section, we evaluate path length of my and Lumelsky's algorithms by the Euclidean distance, and then discuss these advantages against different shaped workspaces. In result, we determine best fit between an workspace and a path-planning algorithm. Strictly speaking, however, it is a difficult problem as the following reason. In general, there are a lot of workspaces with different shapes and in each workspace we can select infinite pairs of start and goal points. Therefore it is difficult for us to find an universal advantage of each algorithm for all cases. Observation of this, we classify workspace shape into the following three pattern:

(1) A simple shape workspace, for example, a workspace with convex shaped obstacles.

(2) A dispersed workspace with complex shaped obstacles

(3) A complicated workspace with complex shaped obstacles

and then we discuss the algorithms' fitness for the three workspaces in average. The average means that one algorithm makes a shorter path in comparison with the other algorithms in most of all workspaces with the same pattern.

First of all, when the robot avoids a convex obstacle, we discuss its path length locally for three typical algorithms and Lumelsky's algorithms. For the convex obstacle, my algorithms Class1, Class2 and Class3 basically make different shorter paths locally. On the other hand, Lumelsky's algorithm Bug2 makes a longer path, and also the algorithm Bug1 makes the longest one locally for the obstacle. To measure the difference for some convex obstacle, we use special shaped obstacles, i.e., a circle and a line. For a circle, my algorithms Class1, Class2, and Class3 generate the shortest path whose length L_{C1}, L_{C2}, and L_{C3} are calculated by $L+(0.75\pi-\sqrt{2})r$, Lumelsky's algorithm Bug2 generates a longer path whose length L_{B2} is done by $L+(\pi-2)r$, and the algorithm Bug1 does the longest path whose length L_{B1} is measured by $L+(2.5\pi-\sqrt{3})r$ as shown in Fig.5(a). The Euclidean distance between start and goal points is denoted by L, and the radius of the circle is denoted by r. Moreover for a line, Lumelsky's algorithm Bug1 makes the longest path whose length is of L_{B1}=SH+HL+LR+RH+HR +RL2+L2G, the algorithm Bug2 makes a longer path whose length is of L_{B2}=SH +HL+LL1+L1G ($L_2L_1 \le HR$, $L_1G \le L_2G+L_2L_1$). My algorithm Class1 makes a similar path whose length is of L_{C1}=SH+HL+LL2+ L2G ($LL_2 \le LL_1$, $L_2G \le L_1G$), the algorithm Class2 makes a shorter path whose length is of L_{C2}=SH+HL+LL3+L3G ($LL_3 \le LL_2$, $L_3G \le L_2G$), the algorithm Class 3 makes the shortest path whose length is of L_{C3}=SH+HL+LG ($LG \le LL_3+L_3G$) (See Fig.5(b)).

An arbitrary convex obstacle can be regarded as a medium shape between the circle and the line, and therefore to avoid the obstacle, these algorithms generate different paths whose length is arranged in the above order (Fig.5(c)). This character is ascertained by the following experiment 1.

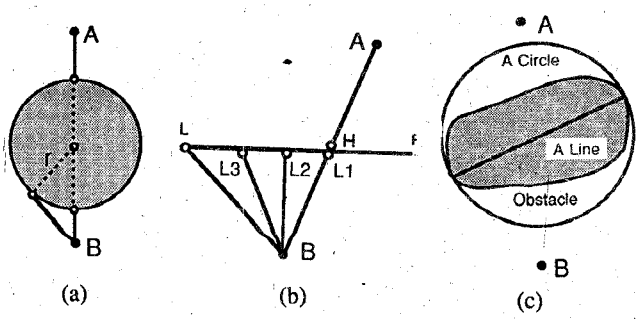

Fig.5 The five paths to avoid a convex obstacle by my and Lumelsky's algorithms for a circle (a), a line (b), and a general obstacle (c)

Experiment 1.

This experiment shows that the order of path length of five algorithms is represented by the inequalities $L_{C3} \leq L_{C2} \leq L_{C1} < L_{B2} < L_{B1}$ in a simple shaped workspace (See Fig.6 and Table.1). This is a regular workspace and the results are useful in practice.

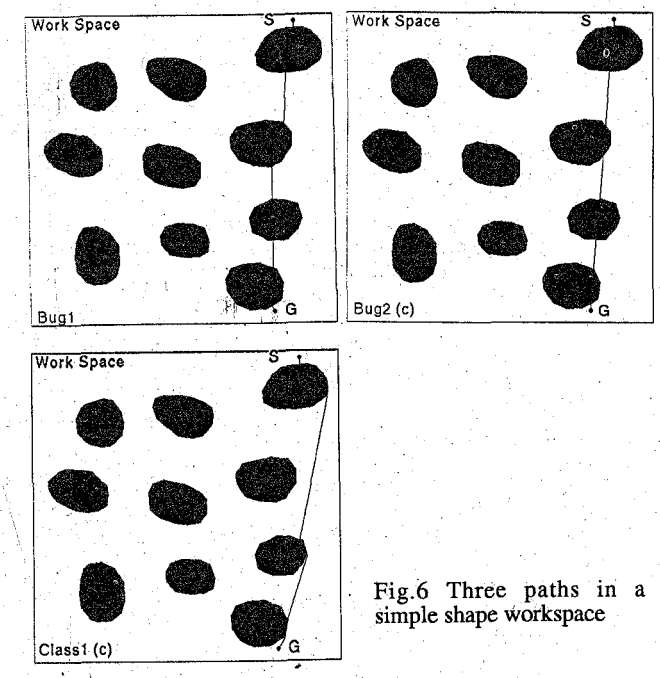

Fig.6 Three paths in a simple shape workspace

	Lumelsky's algorithms	My algorithms			
	L_{B1}	L_{B2}	L_{C1}	L_{C2}	L_{C3}
Clockwise	1872.33	637.15	541.33	542.31	542.31
Counter-clockwise		1034.54	690.26	690.26	690.01

Table 1. All path length by my and Lumelsky's algorithms in a simple shape workspace

	Lumelsky's algorithms	My algorithms			
	L_{B1}	L_{B2}	L_{C1}	L_{C2}	L_{C3}
Clockwise	1046.25	721.46	4027.31	4027.38	31856.57
Counter-clockwise		952.42	396.82	396.82	396.82

Table 2. All path length by my and Lumelsky's algorithms with some loop in a simple shape workspace

Exceptionally, almost all algorithms except the algorithm Bug1 have an undesirable possibility such that the robot collides with an obstacle twice or many times and consequently a generated path includes a loop. In this case, the length order may be reversed. Especially if free space between two obstacles is to be tight in a workspace, the order is completely reversed and my algorithms make a very long path. If we evaluate the path length in the worst case, we must address this bad evaluation for my algorithm. However in a practical situation, such an undesirable case happens uncommon, and evaluation of the path length in the worst case is meaningless. Even in such a case, the algorithm Bug2 keeps the number of leave points small enough, and therefore it does not make a very long path rather than my algorithms. These characters are shown in the experiment 2 as follows.

Experiment 2.

In this workspace, the robot may collides with an obstacle many times in one of clockwise and counter-clockwise orders by my algorithms and Lumelsky's algorithm Bug2, and consequently these paths become longer (Fig.7 and Table 2). Needless to say, such a case happens rarely in a practical situation.

Even in a dispersed workspace with complex shaped obstacles, we can basically discuss the same way, and consequently we accept a similar evaluation of the path length.

Experiment 3.

In an usual dispersed workspace, the robot does not collide with an obstacle twice by my algorithms and Lumelsky's algorithm Bug2, and consequently all algorithms generate their paths whose length is evaluated by the inequalities $L_{C3} \leq L_{C2} \leq L_{C1} < L_{B2} < L_{B1}$ (Fig.8 and Table.3).

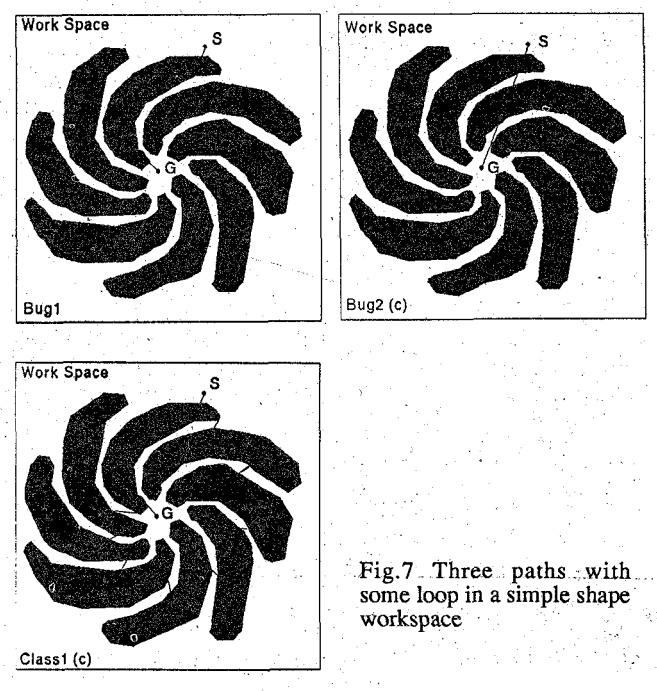

Fig.7 Three paths with some loop in a simple shape workspace

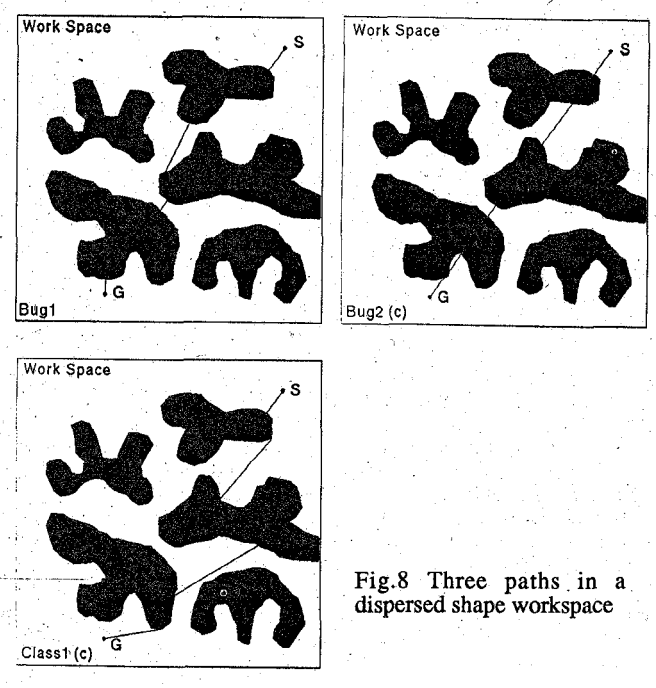

Fig.8 Three paths in a dispersed shape workspace

	Lumelsky's algorithms	My algorithms			
	L_{B1}	L_{B2}	L_{C1}	L_{C2}	L_{C3}
Clockwise	3114.71	1236.52	1020.63	990.03	1169.03
Counter-clockwise		1476.03	1073.76	1065.80	1100.86

Table 3. All path length by my and Lumelsky's algorithms in a dispersed shape workspace

By using the algorithm Bug1, the mobile robot leaves each obstacle from the closest point toward the goal, and therefore the robot never return the obstacle again. However by using the other algorithms, the robot may return the obstacle again and consequently causes some loop in the workspace. Especially, some loop may occurs concerning to a complicated shaped obstacle. In this case, Lumelsky's algorithm Bug2 makes a long local path, and moreover my algorithms make longer local paths because they have a disadvantage such that a lot of leave points are flexibly selected around the obstacle boundary.

Experiment 4.

If a pair of start and goal is badly selected in an uncommonly complicated obstacle, the robot collides with the obstacle many times and its path length becomes longer in all algorithms except for the algorithm Bug1. Thus the algorithm Bug1 is usually good in a workspace with the obstacle. However in this case, the path is not short since the robot always goes round the obstacle in the algorithm Bug1. As mentioned previously, the algorithm Bug2 controls a small number of leave points, and consequently it generates a small number of loops. In result, it is not so bad in such a workspace (Fig.9 and Table 4).

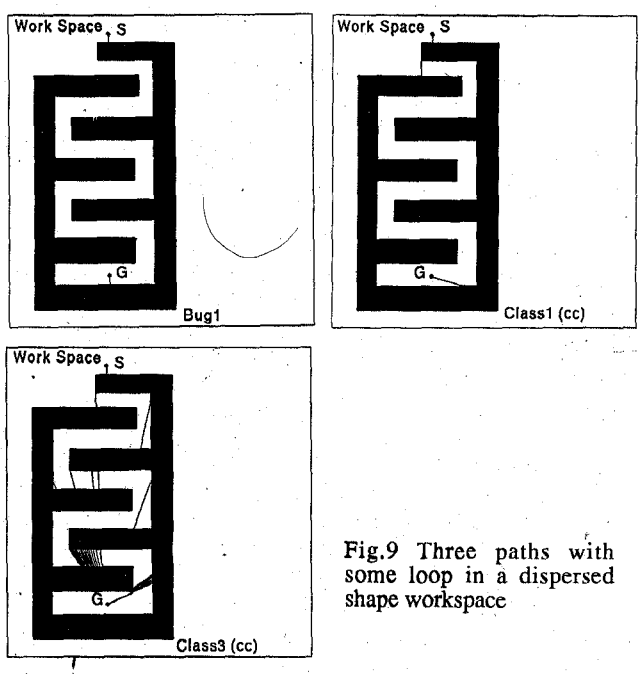

Fig.9 Three paths with some loop in a dispersed shape workspace

	Lumelsky's algorithms	My algorithms			
	L_{B1}	L_{B2}	L_{C1}	L_{C2}	L_{C3}
Clockwise	4804.11	2297.14	2281.24	6481.00	9111.84
Counter-clockwise		2799.97	2524.42	7565.81	88780.92

Table 4. All path length by my and Lumelsky's algorithms with some loop in a dispersed shape workspace

Finally in a complicated workspace with complex shaped obstacles, some loop is easily generated in comparison with the other shaped workspaces. Thus in general, the algorithm Bug1 is robust though it cannot make a very short path. In addition, if we allow a small number of loops in the generated path, my algorithm Class1, Class2 or Lumelsky's algorithm Bug2 is also comfortable.

Experiment 5.

In a complicated shaped workspace, we compare the five algorithms' path length each other. In this case, the algorithm Bug1 does not always make a good path, or rather the algorithm Bug2 and the algorithms Class1 and Class2 are good. The algorithm Class3 is bad. These properties are illustrated in Fig.10 and Table 5.

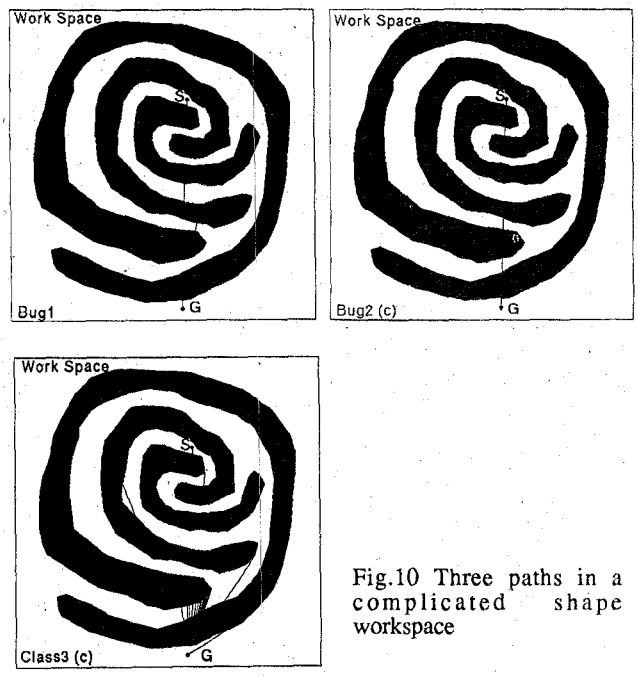

Fig.10 Three paths in a complicated shape workspace

	Lumelsky's algorithms	My algorithms			
	L_{B1}	L_{B2}	L_{C1}	L_{C2}	L_{C3}
Clockwise	7956.78	3784.63	4734.96	4713.24	34847.55
Counter-clockwise		3412.95	3025.67	3063.30	7817.01

Table 5. All path length by my and Lumelsky's algorithms in a complicated shape workspace

In these experiments, we use a two-dimensional digital workspace (image) [0-511,0-511] to represent free shaped obstacle flexibly. In the workspace, the path length is defined by summarizing the following distances:

1) the Euclidean distance is used if the robot goes toward the goal;

2) the digital distance is used when the robot traces an obstacle boundary.

In the digital workspace (image), the digital distance between vertical, horizontal, up, or down neighbor pixels is defined by the Euclidean distance one, and the digital distance between diagonal neighbor pixels is defined by the Euclidean distance square route two.

We should note that the path length differs vehemently if the robot traces the obstacle boundary in clockwise or counter-clockwise order. By this reason, the path length with two orders is respectively evaluated in all experiments.

V. CONCLUSION

Even in an unknown workspace, the mobile robot must arrive at the goal surely while avoiding the obstacles. In this paper, we find a sufficient condition to design such a path-planning algorithm in the uncertain workspace. Based on the sufficient condition, we can construct a lot of deadlock-free and collision-free algorithms with several characters and some of them are better than Lumelsky's algorithms with respect of the path length.

First of all, we show that an algorithm based on the sufficient condition makes a deadlock-free and collision-free path in every workspace and we propose three typical path-planning algorithms running in the uncertain workspace. Moreover, we classify shape of the robot workspace into three types, that is, simple shape, dispersed shape, complicated shape, and then we investigate flexibly fitness between my and Lumelsky's algorithms and such shaped workspaces. As a result, my algorithms are to be better in a simple shaped unknown workspace and also a dispersed shaped unknown workspace because they make more smooth paths after obstacle avoidance without generation of some loop in these workspaces. On the other hand, Lumelsky's algorithms is to be better on an average in a complicated shaped uncertain workspace since they does not always generate a great many loops in the workspace. These characters of my and Lumelsky's algorithms are ascertained by geometrical properties and experimental results.

Acknowledgments

This work was supported by TGL Inc. The author would also like to thank Mr. S.Wazumi for his contribution to this effort.

REFERENCES

[1] T. Lozano-Perez and M.A. Wesley, "An algorithm for planning collision-free paths among polyhedral obstacles," *Commun. ACM*, vol.22, pp.560-570, 1979.

[2] R.A. Brooks, "Solving the find-path problem by good representation of free space," *IEEE Trans. on Systems, Man and Cybernetics*, vol.13, pp.190-197, 1983.

[3] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *The International Journal of Robotics Research*, vol.5, pp.90-98, 1986.

[4] S. Kambhampati and L.S. Davis,"Multiresolution path planning for mobile robots," *IEEE Journal of Robotics and Automation*, vol.2, pp. 135-145, 1986.

[5] V.J. Lumelsky, "Algorithmic and complexity issues of robot motion in an uncertain environment," *J.Complexity*, vol.3, pp.146-182, 1987.

[6] V.J. Lumelsky and A.A. Stepanov, "Path-planning strategies for a point mobile automaton moving admist unknown obstacles of arbitrary shape," *Algorithmica*, vol.2, pp.403-430, 1987.